Universität Rostock



SQL-DDL AUS DEM ER-MODELL ABLEITEN

Joern Ploennigs · AI4SC

Midjourney: The creation of SQL, ref. Michelangelo

Hörsaalfrage

Wenn ein KI-Tool aus einem ER-Diagramm SQL erzeugt: Woran erkennen Sie, ob die Tabellen wirklich korrekt sind?


© Midjourney: Construction plan of the tower of babel

Ziel der Sitzung

- ER-Modell als Spezifikation lesen
- `CREATE TABLE` und `INSERT` als Implementierung einordnen
- KI-generierte DDL systematisch verifizieren
- Fehlende Constraints als Datenqualitätsrisiko erkennen

Rückbezug: `12c_Datenbanken_Entwurf` liefert die Struktur, `08b_CodeReview` liefert das Checklisten-Denken.

Was wird hier festgelegt?

`DDL = Data Definition Language`

- `CREATE TABLE`: Struktur anlegen
- Spaltenname + Datentyp festlegen
- `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE` definieren
- Integritätsregeln in die Datenbank einbauen

Die DDL ist die technische Umsetzung des ER-Entwurfs.

Minimale Form von `CREATE TABLE`

```sql
CREATE TABLE table_name (
    column_name data_type constraints,
    ...,
    table_constraints
);
```

- Jede Entität aus dem ER-Modell braucht eine Tabelle
- Jedes Attribut braucht eine passende Spalte
- Beziehungen erscheinen später als Fremdschlüssel

Datentypen in SQLite

SQLite	Typische Bedeutung
`INTEGER`	Ganzzahl, oft Schlüssel
`REAL`	Gleitkommazahl
`TEXT`	Text
`BLOB`	Binärdaten
`NULL`	fehlender Wert

KI kann passende Typen vorschlagen. Sie prüfen, ob sie zur fachlichen Bedeutung des Attributs passen.

Beispiel: KI-generierte DDL für Geometrie

```sql
CREATE TABLE Points (
    point_id INTEGER PRIMARY KEY AUTOINCREMENT,
    x REAL NOT NULL,
    y REAL NOT NULL
);

CREATE TABLE Lines (
    line_id INTEGER PRIMARY KEY AUTOINCREMENT,
    start_point INTEGER NOT NULL,
    end_point INTEGER NOT NULL,
    FOREIGN KEY(start_point) REFERENCES Points(point_id),
    FOREIGN KEY(end_point) REFERENCES Points(point_id)
);
```

Frage vor dem Ausführen: Entspricht das wirklich dem ER-Modell?

Hörsaalfrage

Die KI hat die Tabellen `Points` und `Lines` erzeugt. Arbeiten Sie zu zweit:

- Nennen Sie einen Aspekt, der sicher korrekt wirkt.
- Nennen Sie einen Aspekt, den Sie gegen das ER-Modell prüfen müssten.
- Welche Constraint könnte fehlen, obwohl das SQL formal gültig ist?

DDL-Verifikationscheckliste I

Prüfen Sie jede KI-generierte Tabelle gegen das ER-Diagramm:

- ☐ Jede Entität -> eigene `CREATE TABLE`
- ☐ Jedes Attribut -> passende Spalte
- ☐ Datentyp fachlich plausibel
- ☐ Primärschlüssel vorhanden

Wenn eine Entität oder ein Attribut fehlt, ist das kein Syntaxfehler, sondern ein Modellfehler.

DDL-Verifikationscheckliste II

- ☐ Jede Beziehung -> `FOREIGN KEY`
- ☐ Pflichtattribute / Kardinalitäten -> `NOT NULL`
- ☐ Eindeutigkeit -> `UNIQUE`
- ☐ Mehrdeutige KI-Annahmen dokumentieren oder nachprompten

Merksatz: Ein Constraint, der nicht in der DDL steht, wird auch nicht durchgesetzt.

Agentischer Workflow: ER -> SQL -> Review

> **Wenn KI `CREATE TABLE` erzeugt:** 1. Lesen Sie zuerst das ER-Diagramm, nicht den SQL-Text. 2. Gehen Sie die Checkliste Punkt für Punkt durch. 3. Markieren Sie fehlende `NOT NULL`-, `UNIQUE`- oder `FOREIGN KEY`-Regeln. 4. Ergänzen Sie den Prompt: “Erzeuge die DDL erneut und übernimm alle Pflichtattribute und Eindeutigkeitsbedingungen aus dem ER-Modell.”

Das ist dieselbe Arbeitsweise wie in `08b_CodeReview`: Spezifikation gegen Artefakt prüfen, nicht der KI vertrauen.

Daten einfügen mit `INSERT`

```sql
INSERT INTO Points(x, y) VALUES(54.083336, 12.108811);
INSERT INTO Lines(start_point, end_point) VALUES(1, 2);
```

- `INSERT` füllt nur Tabellen, es definiert sie nicht
- Fremdschlüsselwerte müssen auf existierende Datensätze zeigen
- Fehler beim Einfügen sind oft ein Hinweis auf fehlende oder korrekte Constraints

Tabelle aus vorhandenen Daten erzeugen

```sql
CREATE TABLE PointsCopy AS
SELECT * FROM Points;
```

- praktisch für Zwischenergebnisse oder Materialisierung
- aber: Primärschlüssel und weitere Constraints werden nicht automatisch vollständig übernommen

Auch hier gilt: Nach dem Erzeugen wieder gegen die Checkliste prüfen.

SQL ist nicht universell: Netzwerke

Für manche Informationsmodelle passt ein relationales Schema nur schlecht:

- Straßennetze oder Leitungsnetze sind vor allem **Knoten + Kanten**
- typische Fragen: Nachbarn, Wege, Erreichbarkeit, kürzeste Route
- dafür sind **Graphdatenbanken** oft das passendere Modell

Nicht jede Struktur sollte in Tabellen gepresst werden.

SQL ist nicht universell: BIM und Dokumente

- BIM-Elemente besitzen oft viele optionale, verschachtelte Eigenschaften
- Dokumente und JSON-ähnliche Strukturen ändern sich häufig
- dafür sind **Dokumentdatenbanken** oft flexibler als starre Tabellenschemata

Ingenieurprinzip: Erst Informationsmodell verstehen, dann Datenbanktyp wählen.

`ALTER TABLE` und `DROP TABLE` nur als Referenz

- `ALTER TABLE`: bestehende Tabellen später ändern
- `DROP TABLE`: Tabellen löschen
- Schema-Evolution ist wichtig, aber heute nicht unser Fokus

```sql
ALTER TABLE Points RENAME TO Punkte;
DROP TABLE IF EXISTS Points;
```

Für diese Sitzung reicht: Begriffe kennen, nicht im Detail beherrschen.

Was Sie mitnehmen sollten

- Das ER-Modell ist die Spezifikation, die DDL ist die Implementierung
- KI darf `CREATE TABLE` formulieren, Sie prüfen es gegen das Modell
- Fehlende Constraints sind echte Datenqualitätsrisiken
- SQL ist stark für relationale Daten, aber nicht für jedes Informationsmodell

QUESTIONS